

Introduction

Nowadays we have a tui for everything, but there's somehow no tui grooveboxes (a self-contained electronic music production device that combines multiple components in one unit) on the market (I did a quick search on google and <https://terminaltrove.com/> and I found nothing, so i guess we are the only one...)

So our goal for the final project is to build a networked collaborative TUI sequencer: each user runs a terminal instrument similar to the Teenage Engineering Pocket Operator (a lightweight groove box with built in synth, drums, bass sounds) and jams together over a shared session.

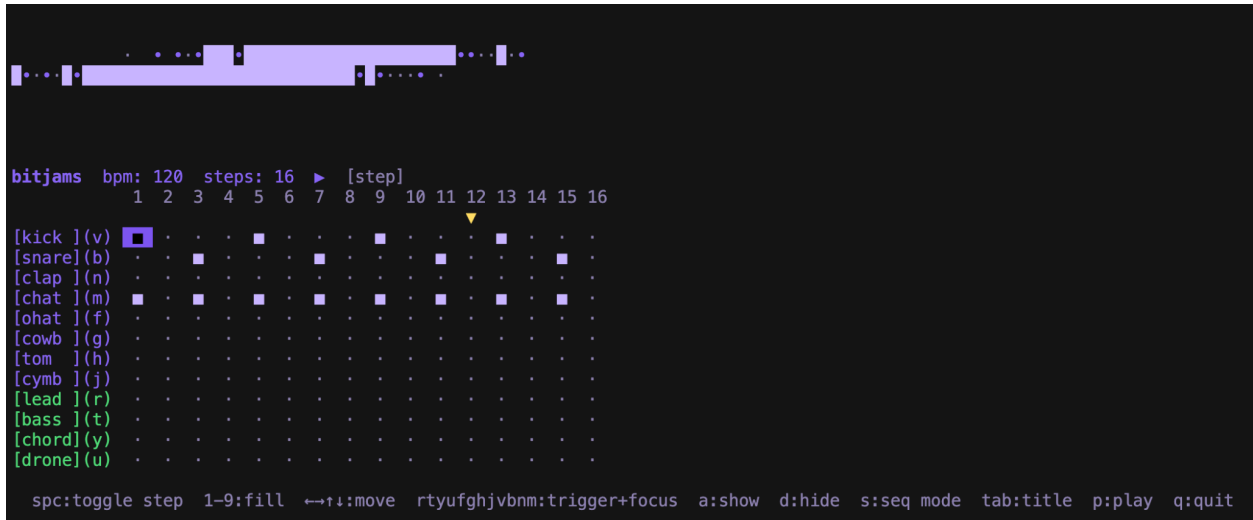


Design/Implementation

Tools and stack:

The prototype is written in C++, built with CMake and linked against Apple's CoreAudio frameworks on macOS. Audio I/O is handled by miniaudio, a single-header C library that wraps platform audio APIs and drives the synthesis callback. The terminal interface uses raw ANSI escape codes directly although we might switch to a tui library later.

We built a TUI Digital Audio Workstation in C++ with multiple drum voices and one minimal synth, running entirely in the terminal with no external assets. The interface renders as a 16-step sequencer grid, users move a cursor across steps with arrow keys and toggle cells with spacebar. A playhead advances across the grid at a fixed BPM, and a loop counter tracks how many times the pattern has cycled.



All drum sounds are synthesized procedurally at runtime via a callback-based audio engine using miniaudio. The kick is a sine wave with a pitch envelope sweeping from 180 Hz to 40 Hz over ~30ms, decaying over 300ms. The snare layers white noise with a 200 Hz square wave and a short high-pass bias. The hi-hat is band-passed noise with a sharp 50ms decay. The clap is three staggered noise bursts 8ms apart with a transient click at onset. To achieve a lo-fi, 8-bit aesthetic consistent with the PO aesthetic, all voices are quantized to 8-bit resolution before mixing to get a retro sound, each sample is cast through `int8\_t` and back, crunching the waveform to 256 levels.

Disregarding the networking, the system runs three threads: a main thread handling raw terminal input and redrawing the screen at ~30 fps, a timing thread that fires every 125ms to advance the step counter and trigger voices, and miniaudio's own audio callback thread which mixes active voices into the output buffer. The timing and audio threads communicate through a small set of atomic flags to avoid locks on the hot path. Terminal raw mode is installed at startup and restored in an `atexit` handler so a crash does not leave the shell broken.

A stereo visualization is rendered alongside the grid as a live Lissajous-style stereogram, giving performers a visual read on the current mix.

Synchronizing sessions:

A core challenge of networked collaborative music is that audio is far more sensitive to timing irregularities than general data transfer. Even small delays produce flammings — the audible artifact of notes that were intended to land together arriving slightly apart. Any sync model must confront this directly.

Our approach to this was to have each session be responsible for generating its own audio, but have individual hosts be responsible for transmitting updated state information over TCP. On startup, a user is prompted to select their option, which includes starting a new public session, starting a new private session, connecting to a server, opening the help menu, and quitting the application.

Starting with the simple commands, the private session is a personal session that other users are not allowed to connect to. Quit leaves the application and closes all threads. Help opens the help menu, showing all available repl commands.

On selecting a public session, a room number gets assigned to the session, which must be unique compared to other open room numbers on that host (if they have several sessions open). The host then opens a thread listening for incoming connections on a fixed port for the entire application (1234). When a client wants to connect, they enter the IP address and the room number and an attempt is made to connect to the server. Whether this attempt is successful or not depends on whether or not the server can find the desired room number in its map. If it does, the state of the session is sent over the network to the client so it can initialise itself with the host's session.

After establishing a connection, the client and server rely on bit-packed highly optimised diff structure to send updates. With every update to the session state, the diff struct gets updated and on a regular basis (set to 50ms), the diff struct gets sent over the network. More specifically, the host polls its clients and after the time limit is up, it receives all of their diff structs and resolves any collisions using RNG. After collisions are resolved, the host updates its personal state and sends back a diff to all of the clients so that they all have the same shared state. This being said, clients do not know about other clients, making our implementation server-based rather than P2P.

Assumptions:

The scope of our project was quite ambitious, so we ended up making some assumptions and compromises. One of these is that all users must be on the same local network. This is because we are not doing DNS or any other higher level addressing, so individual devices are not visible.

Discussion/Results:

Tests:

- Tests disregarding real-time synchronization:
 - Connecting to a non-existent room fails as expected
 - Starting private session does not expose room number as expected
 - Starting new public session enables user on localhost to connect and receive current state of host
 - Joined host session, received correct state. Disconnected, updated the hosts session, then reconnected and it still sent the correct state as expected.
 - Quit host session, user session stayed open as expected
 - After quitting the host session, user session quit and tried to reconnect, which resulted in a failure as expected
- Tests with synchronization:
 - Editing on host side changes on client
 - Editing on client side changes on host

AI use:

We used AI in two ways: one was with online LLMs to support writing manual code; and the other was using Cursor. We made sure to have a good idea of architecture, which we implemented the general infrastructure for manually. Afterward, we passed on the codebase to cursor, which asked us questions about design decisions and made us think more deeply about synchronization and edge cases (like what to do when a host exits and a user stays online). We noticed it asked some dubious questions and offered strange suggestions that would not change much.

- Networking is hand written with AI assistance for ideation and generating smaller helpers (like boilerplate code for generating a socket)
- DSP code was partially handwritten, with some instruments being completely AI generated
- UI / visualizers are mostly AI generated to save time given the scope of the project

Overall, our project was a success. We found there to be low latency locally and with the network portion.

Conclusions/Future work:

Overall, we learned that sharing state is a very involved problem with many design decisions. We spent a good deal of time discussing whether we should send raw state, or diffs, or something in between with bitpacking. The issue with sending the whole state, bitpacked or not, is that a change made locally can get overwritten by the host and has to get acknowledged first.

In addition to this, we learned that processing that state across several clients offers many other decisions to be made, such as how to resolve collisions and how to ensure that everyone connected to the server including the host itself observes the same state without gradual drift.

For future work, we would make this system work over the internet and not just a single network. We would also add layers of complexity and extensibility to the state, such as enabling users to sculpt the sound choices so that DSP parameters get updated as well and making the project length variable, which is not currently supported by our fixed bitpacking schema (only 16 bits for the time axis).